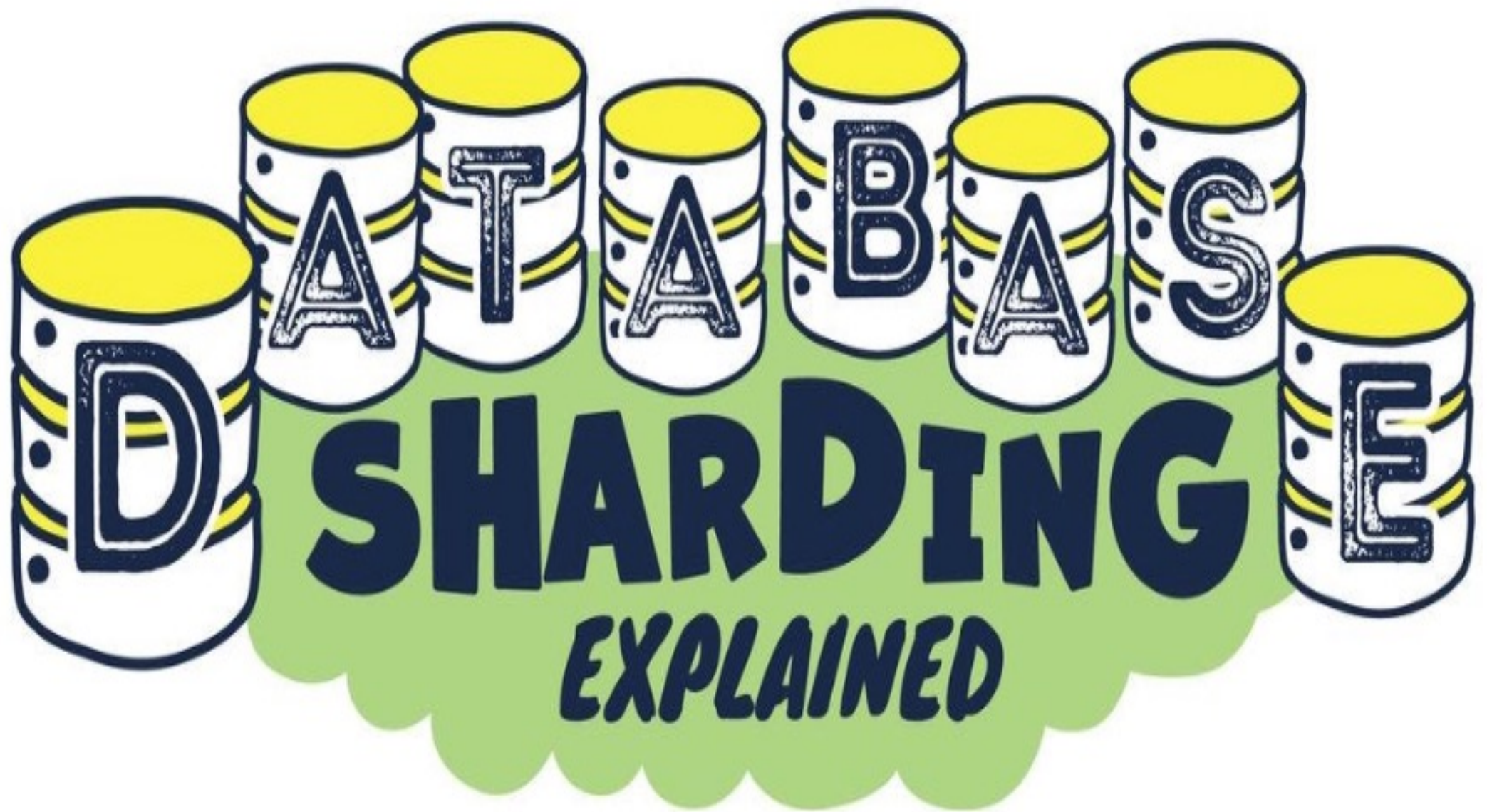
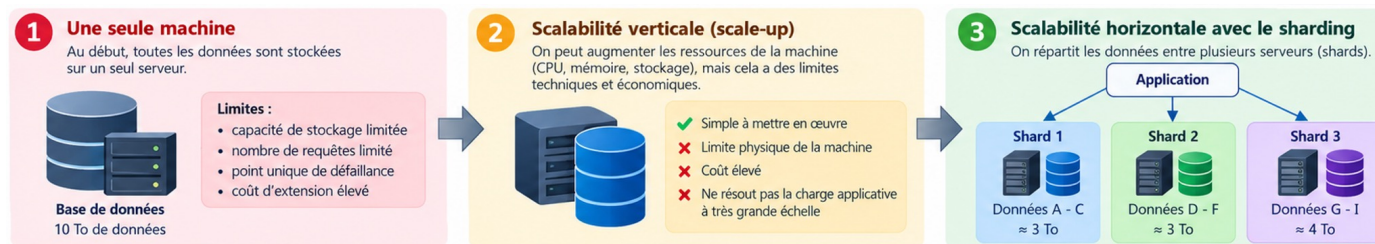




Le sharding



Notre site fonctionne bien avec un Replica Set. Mais le succès arrive : des millions de produits et de clients, des milliards de commandes, plusieurs téraoctets de données et une charge qui continue d'augmenter.

On peut commencer par augmenter les ressources de cette machine : davantage de CPU, de mémoire ou de stockage (scale-up).

Mais cette approche rencontre nécessairement des limites techniques ou économiques.

Une autre stratégie consiste à ajouter plusieurs machines et à répartir les données entre elles.

Le sharding est une technique de partitionnement horizontal qui consiste à répartir un ensemble de données entre plusieurs serveurs, appelés shards.

Chaque shard ne contient donc qu'une partie des données.

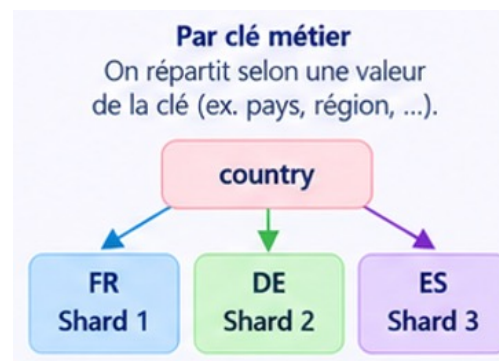
Le sharding



Le sharding ne consiste pas seulement à découper les données. Il faut également disposer d'une stratégie permettant de déterminer où chaque donnée doit être stockée et retrouvée.

Cette stratégie est basée sur la clé de partitionnement, La clé de partitionnement (*partition key* ou *shard key*) est une donnée utilisée par le système pour déterminer sur quel shard une donnée doit être stockée et recherchée.

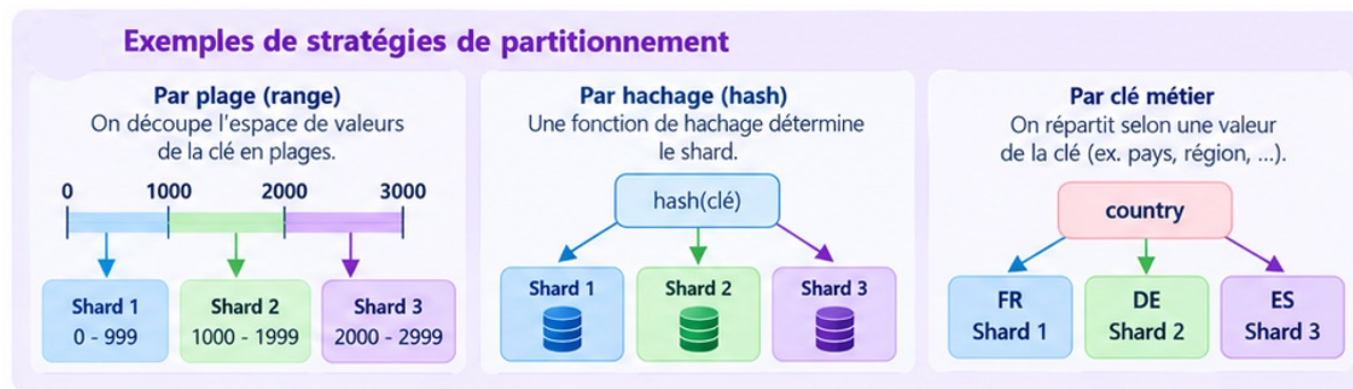
Par exemple :



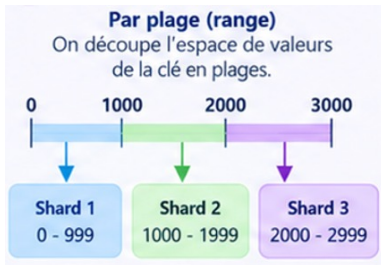
Le sharding

La fonction de partitionnement est la règle ou l'algorithme qui utilise la clé de partitionnement pour déterminer sur quel shard une donnée doit être placée.

La clé indique sur quelle information répartir les données ; la fonction de partitionnement indique comment utiliser cette information pour choisir le shard.



Partitionnement



Le partitionnement par intervalle consiste à diviser les valeurs de la clé de partitionnement en plusieurs plages ordonnées.

Chaque plage est ensuite associée à une partition ou à un shard.



Le partitionnement par hachage consiste à appliquer une fonction de hachage à la clé de partitionnement afin de déterminer où stocker la donnée.

Le partitionnement par hachage privilégie une distribution homogène



Le partitionnement par liste consiste à associer explicitement certaines valeurs de la clé de partitionnement à une partition.

Contrairement au partitionnement par intervalle ou par hachage, la répartition repose ici sur une liste de valeurs prédéfinies.



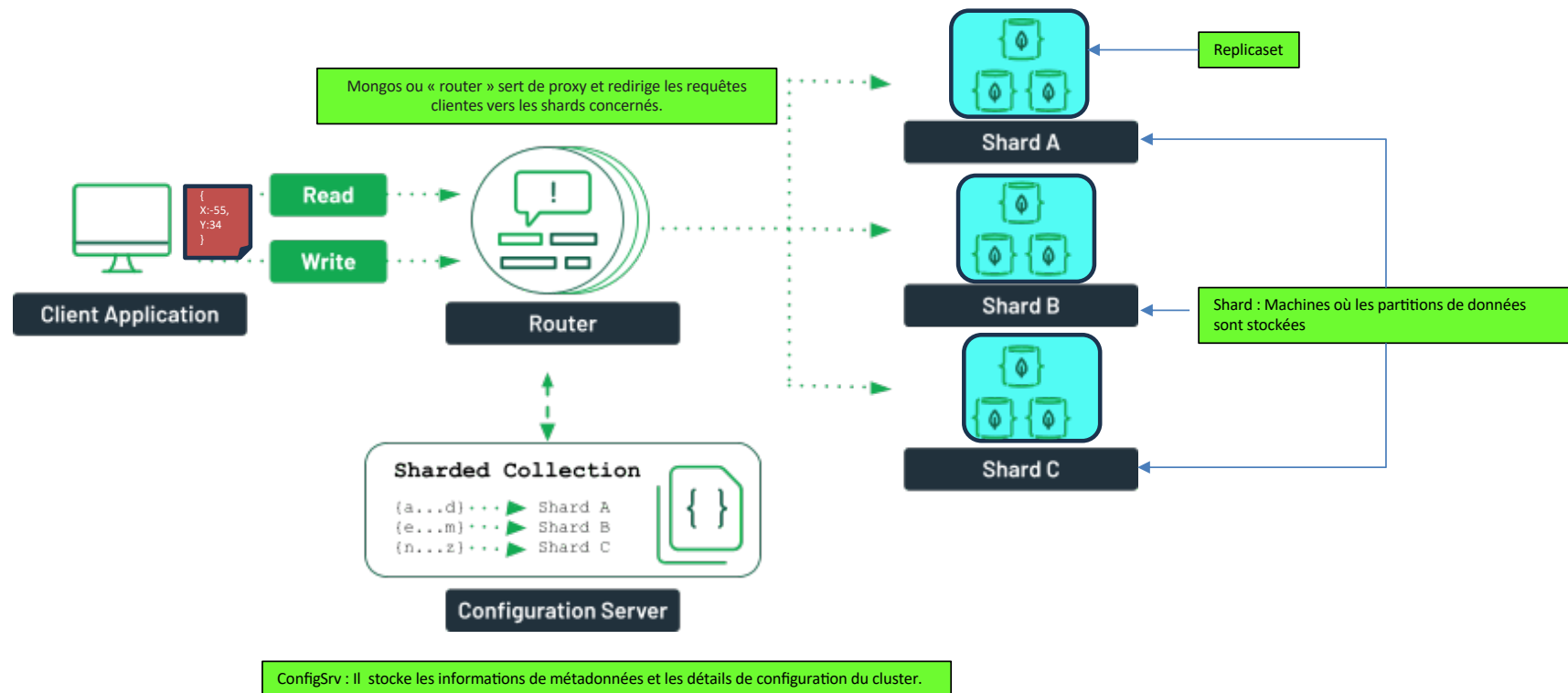
Le choix de la stratégie de partitionnement est déterminant pour les performances et la scalabilité du système.

- Tenir compte des requêtes de l'application
Une requête ciblant un seul shard sera généralement plus efficace qu'une requête nécessitant d'interroger tous les shards
- Choisir une bonne clé de partitionnement
La clé doit permettre une répartition équilibrée des données mais aussi de la charge entre les shards.
- Choisir la stratégie adaptée aux accès
Range pour préserver la localité et faciliter les recherches par intervalle ; Hash pour favoriser une distribution homogène ; List pour contrôler explicitement le placement de certaines catégories.

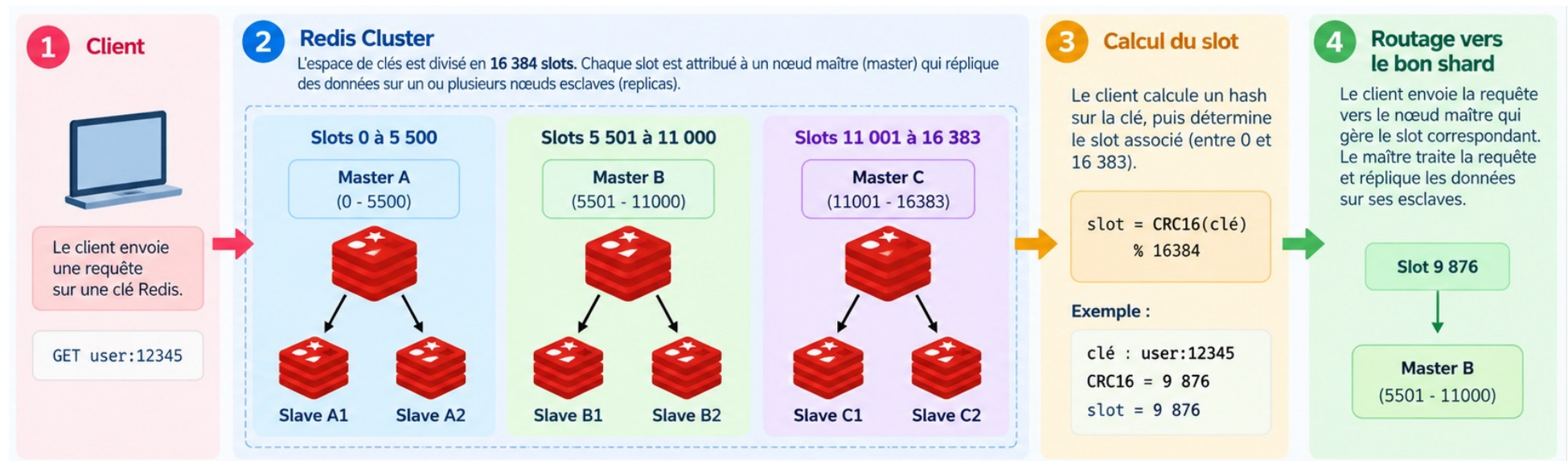


- **Prévoir l'évolution et le rééquilibrage**
Le volume et la charge évoluent. Il faut pouvoir ajouter des nœuds et redistribuer les partitions sans interruption majeure.
- **Ne pas confondre sharding et haute disponibilité**
Le sharding distribue les données et la charge. La réplication fournit la redondance et la tolérance aux pannes. Les deux mécanismes sont généralement complémentaires.
- **Éviter les déséquilibres et les hotspots**
Un shard ne doit pas concentrer la majorité des données, des lectures ou des écritures.
- **Un bon sharding doit répartir les données, répartir la charge et permettre de localiser efficacement les données recherchées.**

Le sharding avec MongoDB



Le sharding avec REDIS



TP 01 Sharding avec REDIS

